WEEK 4

BEHAVIOURAL DIAGRAMS



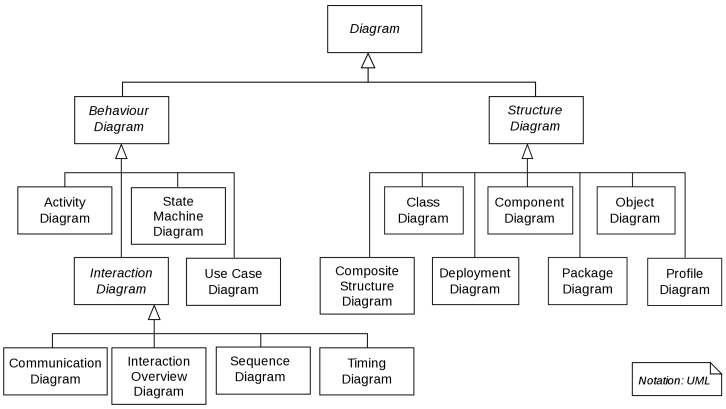
Mr Randell Gaya

Module Originator: Dr Tom Blanchard

TODAY

1. UML behaviour diagrams: what are they?
2. Use case diagrams.
3. State machine diagrams.
4. Sequence diagrams.

UML DIAGRAM TYPES



BEHAVIOURAL DIAGRAMS

Describes the **how** a system works.

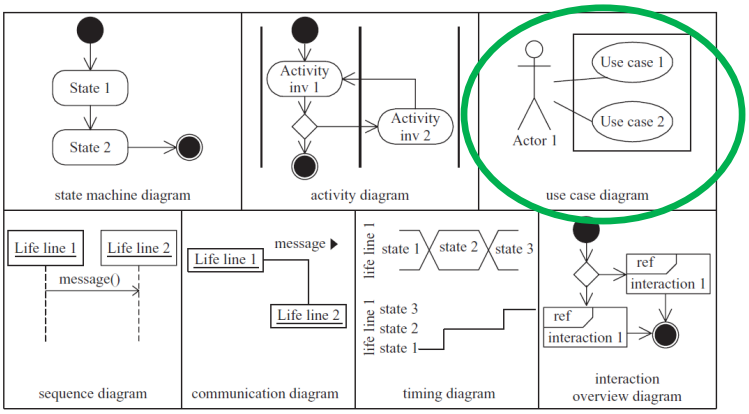
Demonstrates how a system behaves over time by showing:

- The order in which things happen.
- The conditions under which they happen.
- The interactions between things.

USE CASE DIAGRAMS



BEHAVIOURAL DIAGRAMS

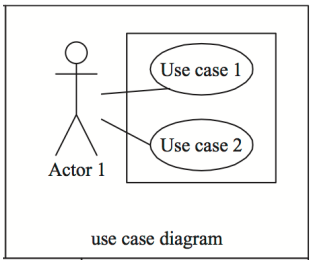


Jon Holt, UML for Systems Engineering: Watching the wheels, IET, 2007.

USE CASE DIAGRAM

Fundamentally:

- Who uses the system.
- How will they use it?



use case diagram

Can essentially represent the complete set of requirements for a system.

USE CASE DIAGRAMS

A simple behavioural diagram (only a few pieces of syntax) used for:

- **Requirements** modelling.
- Business / system **context** modelling.

Fairly intuitive way of **gathering requirements**.

Represent the interaction between various **types of user** and the system.

Good for **communicating** with stakeholders.

Visualises **requirements capture**.

USE CASE DIAGRAMS

Helps us answer some **fundamental** questions!

- **What** should the system do?
- **What** are the **boundaries** and **what** is inside/outside?
- **What** do we need?

Help us identify objects, relationships, classes.

- These will mostly **not** be modelled in the diagram.

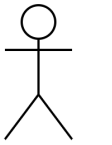
When analyzing/designing a solution to a domain problem…

- Take existing solutions (software or otherwise), requirements, experts in the domain etc.
- Produce a **use case diagram**.

ACTORS

Somebody or something that **interacts** with the system.

- External '**users**' of the system.
- Represented as a stick figure.
- **Logical** not physical users.



Bank Manager

Should relate to **at least one** Use Case.

- If they don't something is wrong!

When identifying Actors think:

- Roles, organization, devices, other systems.

USE CASES

Shown as an oval filled with the use case name.

An action that can be carried out, by Actor(s), **using the system**.

- Multiple actors can be associated with one use case.
- Use cases can be **related** to each other.



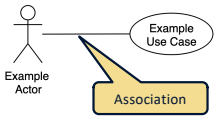
How to identify a use case?

- Think about each actor and what they **need to do**.
- How does 'X' use the system?
- Ask the 'actors'.
- How do we get **information in/out** of the system

RELATIONSHIPS

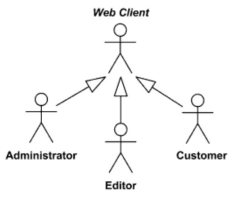
Associations

- The relationship between an Actor and a use case.
- They cross the system boundary and represent interfaces.
- Can add multiplicities.



Specialization / Generalization.

- Same concept /notation we have seen in class diagrams.



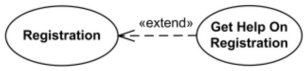
RELATIONSHIPS

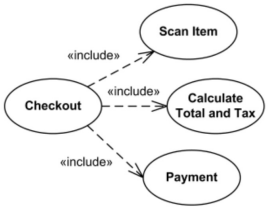
Includes. Used when there are common parts of the behavior of two or more Use Cases.

<<includes>>

Extends. Used when there is some additional behavior that should be added to a Use Case.

<<extends>>



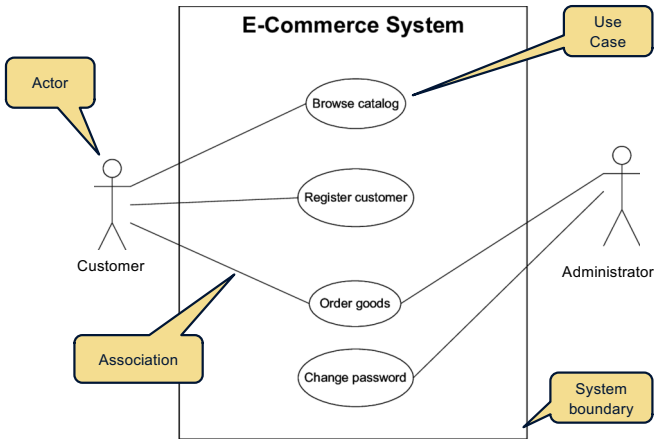


SYSTEM BOUNDARY

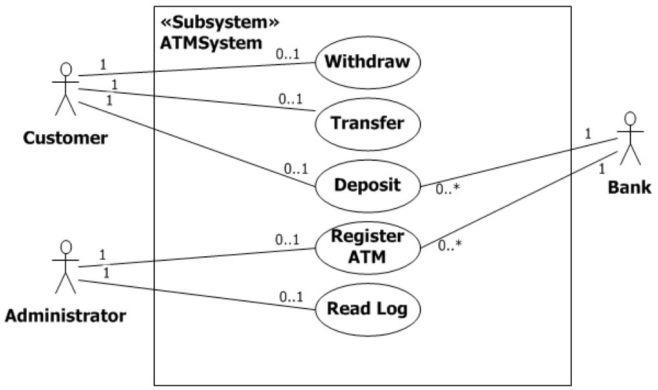
Represents the context that the system exists or operates in.
Important as it allows to visualize what is and isn't part of the system.
Can have a name and stereotype.



EXAMPLE



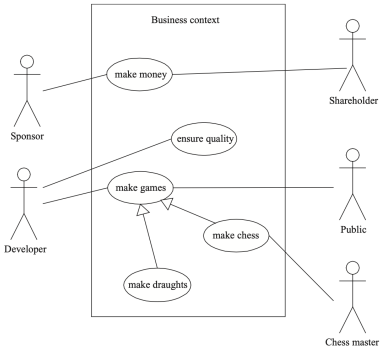
EXAMPLE



BUSINESS CONTEXT

The business requirements of the organization.

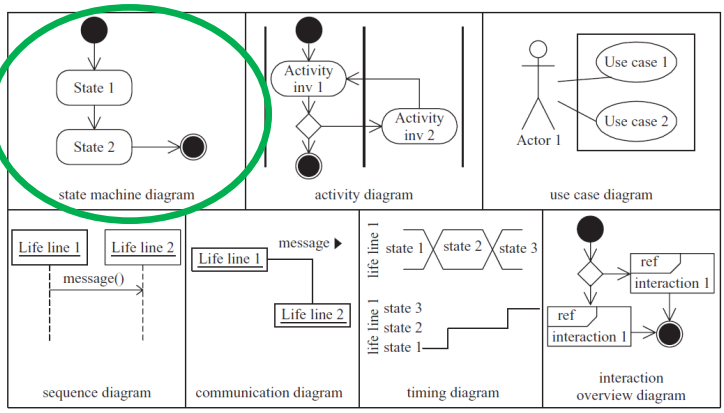
Actors associated with the business requirements.



STATE MACHINE DIAGRAMS



BEHAVIOURAL DIAGRAMS



STATE MACHINE DIAGRAMS

Widely used.

- People are familiar with them conceptually.
- Strong relationship to classes and therefore class diagrams.

A state machine diagram describes the behavior of an object

- Usually covers the objects lifetime (from instantiation to destruction).

RELATIONSHIP TO CLASS DIAGRAM

A class is a template for creating objects

- The **Cat class** defines the attributes a **Cat object** will have and the methods that can be called on it.
- Behavior is part of what defines an object but a class diagram is all about structure.
- How do we represent behavior in a class diagram?

If we want to describe the behavior of objects of a class we need a State Machine diagram.

- Does every class need one?
- **No!**
- Some classes have no behaviors.

COMPONENTS OF A STATE MACHINE DIAGRAM

State

- Describes the situation in the execution during which some invariant condition holds.
- Describes when the system satisfies a particular condition.
- Often relates to attributes. eg ‘loaded’ or ‘saved’.
- Could describe inactivity or waiting.
- Assumed to take a finite amount of time.
- Can be interrupted. (non atomic)



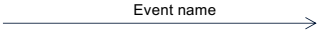
COMPONENTS OF A STATE MACHINE DIAGRAM

Transition

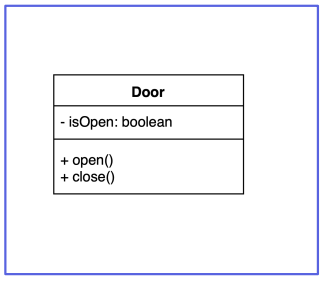
- Connects two states together.
- It is unidirectional.

Event

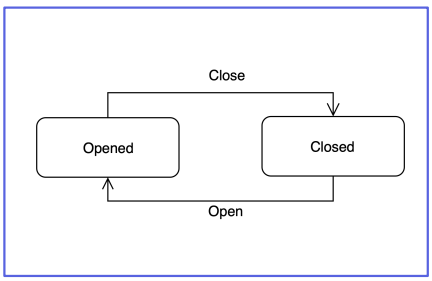
- In order to cross a transition we need an event.
- Represents something happening, starting or stopping.
- Could also represent receiving a message from another part of the system.



EXAMPLE

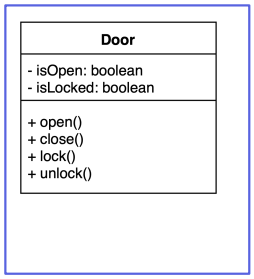


Class Diagram

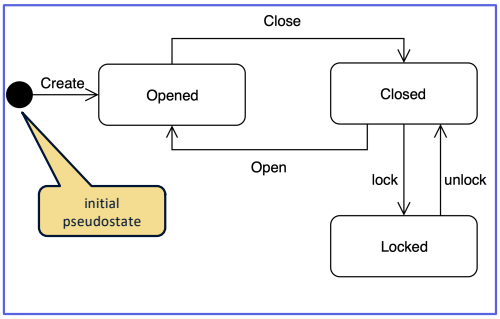


State Machine Diagram

BIGGER EXAMPLE

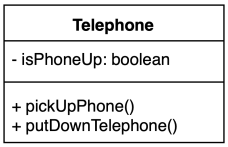


Class Diagram



State Machine Diagram

YOUR TURN

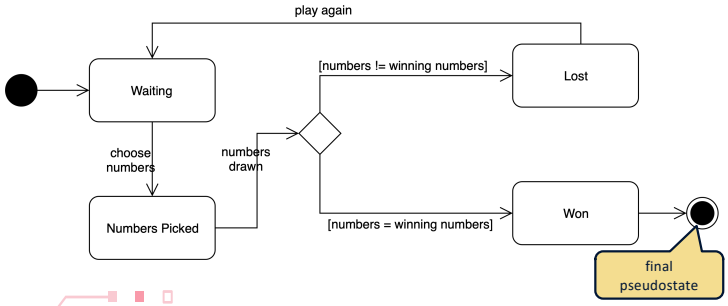


ONE MORE BIT OF SYNTAX...

We can have decisions.

They are represented as unfilled diamonds.

Evaluates the guards on the events that trigger a transition.

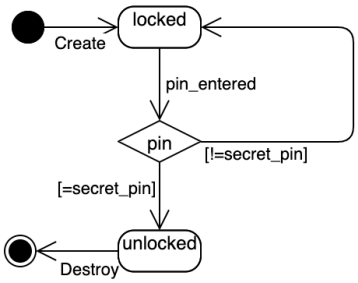


CHOICES

Can put a common left operand inside the choice

Guard constraints are evaluated dynamically.

Split a transition into multiple alternative paths based on some condition.



SUMMARY

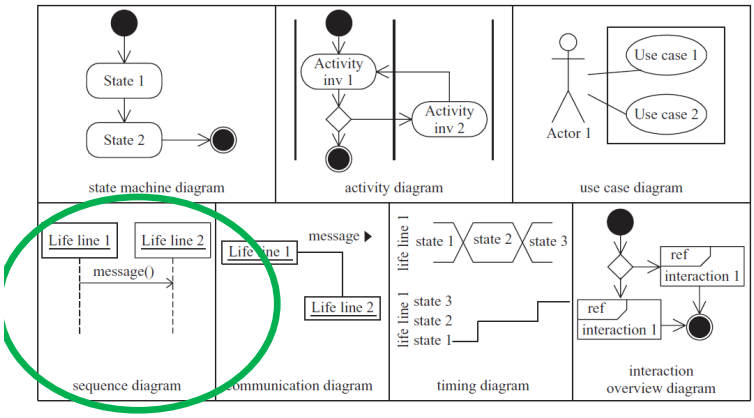
1. Use Cases diagrams **are** useful:
- Focused on **who** can do **what**.
- Where is the system boundary.
- Good for requirements capture.

2. State Machine Diagrams are about showing behaviour:
- The states an object can be in.
- How an object transitions from state to state.
- The events that cause / trigger transitions.
- There is a lot more to State Machine Diagrams

SEQUENCE DIAGRAMS



BEHAVIOURAL DIAGRAMS



Jon Holt, UML for Systems Engineering: Watching the wheels, IET, 2007.

SEQUENCE DIAGRAM

Most widely used type of **interaction** diagram.

- The three others are Activity, Timing and Communication.
- These can all be **combined** into one Interaction diagram.

All about the **interactions** between actors and objects within a system.

Describes the **sequence of messages** between objects.

- *"The most common variant is the Sequence Diagram that focuses on the Message interchange between a number of Lifelines."*

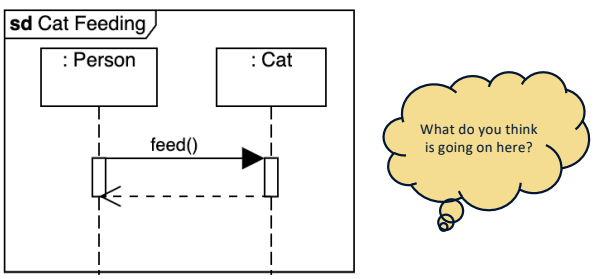
A sequence diagram shows the sequence of interactions that take place during a particular use case or use case instance

SEQUENCE DIAGRAMS

As with all UML diagrams, a formal syntax is provided.

- There is quite a lot of available syntax!

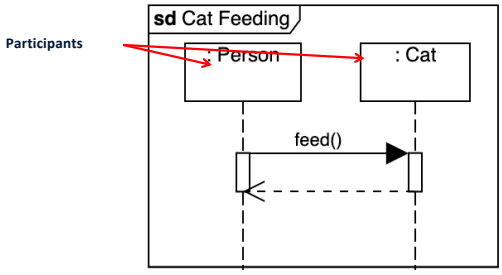
Easy to understand by developers and clients.



READING A SEQUENCE DIAGRAM

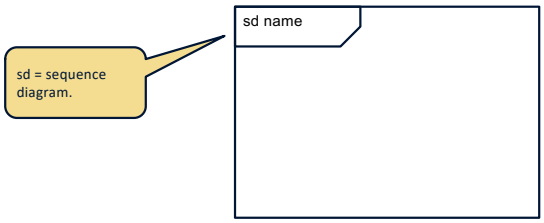
Lead left to right and top to bottom.

- Participants (Timelines) along the top.
- Move further along timelines as you go down the diagram.



FRAMES

Group interactions together. Contains the focal portion of the sequence diagram.
Name of sequence diagram given in a box in the top left in a pentagon.
Not specific to sequence diagrams.



LIFELINES

Also referred to as **participants**.

An individual participant in the interaction.

Often a given instance of a class from a structural diagram.

Reside at the top of the frame.

Named in a rectangle. *(Sometimes underlined but don't do this.)*

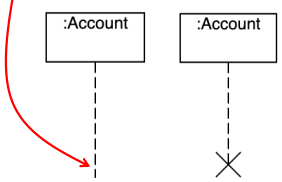
Lifeline Participant **Lifeline Name : Class** **Tibbles : Cat**

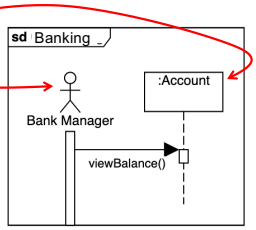
Can omit the name for an anonymous instance.

LIFELINES

Lifelines could be actors or objects.

Lifelines have dashed lines that represent when that participant is 'alive'.





If an object is destroyed during the sequence diagram the lifeline is terminated with a cross.

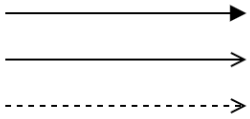
MESSAGES

Some sort of **message** between objects/actors.

- Method calls, signals, creating instances etc.

Arrow-headed lines between lifelines.

- Message call – solid line with a:
- Filled arrowhead (**Synchronous** message) or
- Unfilled arrow (**Asynchronous** message).

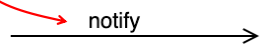


- Message **reply** to call – dashed arrow line.

MESSAGES

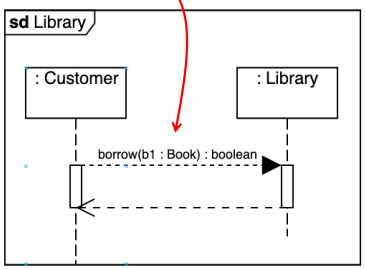
Messages should have **labels** attached to them.

- Could be a simple descriptive name.
- When the relevant information is available you may add it with the same **syntax as an operation** in a class diagram.



During a **synchronous** message the caller / sender cannot do anything until the receiver has finished.

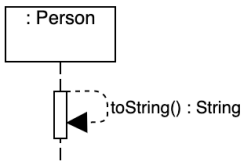
For **asynchronous** messages this is not true.



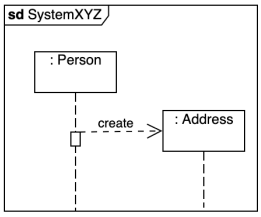
MESSAGES

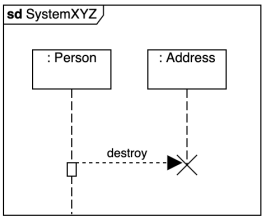
Lifelines can send messages to **themselves**.

- How might this be represented in Java code?



There are also explicit create and delete messages.



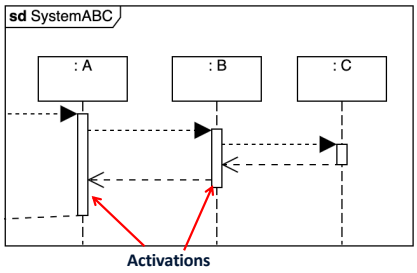


ACTIVATION

Also referred to as Execution Specifications or Control Regions.

Those little white **rectangles** (can also be grey).

- Pass **activation focus** between lifelines.
- That object is 'doing something'; sending signals, executing some behaviour or waiting for a reply.

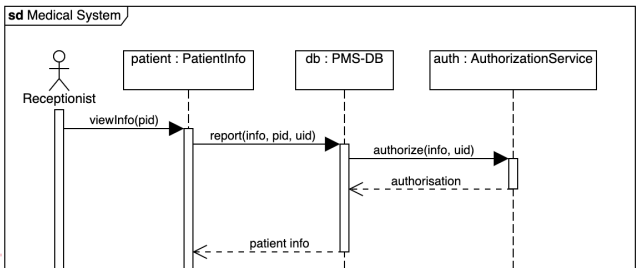


HAVE A GO...

A receptionist sends a **viewInfo** message to a **PatientInfo** object and provides a **pid**.

When the **PatientInfo** object receives this message it sends a **report** message to the **database** and passes the **info**, **pid** and **uid**.

The **database** then sends an **authorize** message to an **authorizationService** and provides the **info** and **uid**. The **authorizationService** returns the **authorization** to the **database** which returns the **patient info** to the **patientInfo** object.



SEQUENCE FRAGMENTS

Allow better organization / structuring of sequence diagrams.

- We can represent loop fragments, conditional fragments, optional fragments, parallel fragments and break fragments etc.
- They can be nested.

Many different types but we will look at:

- **ref** – Refers to another sequence diagram.
- **alt** – Alternative sequence(s) based on some condition.

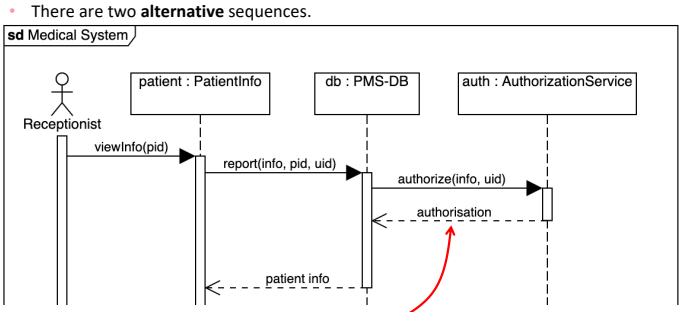
Represented with the same syntax as a frame.

- Pentagon in top left corner.
- Relevant name in the pentagon.

ALT FRAGMENT

The problem with this is that authorization could return true/false.

- There are two **alternative** sequences.

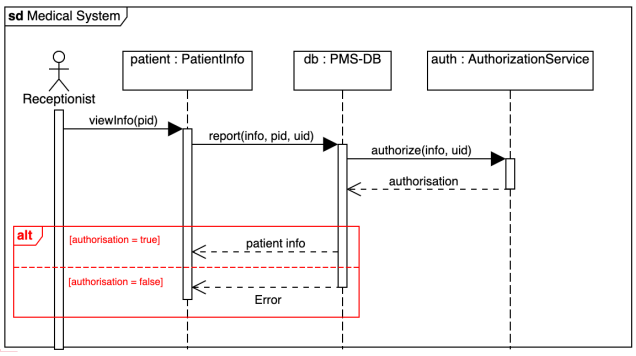


What happens if we are/aren't authorized?

ALT FRAGMENT

Use an alt fragment (shown in red).

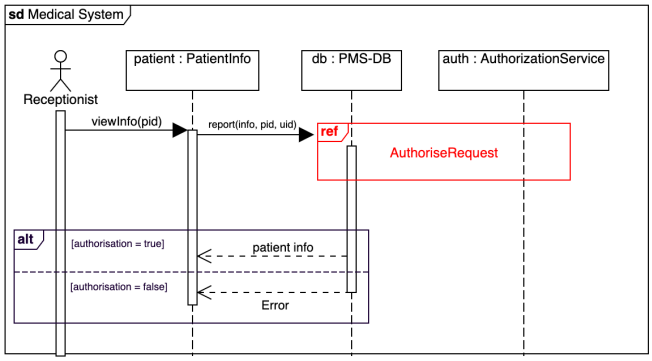
- The sequences that run depends on the guard statements.
- Can provide an [else] statement as a last resort.



ALT FRAGMENT

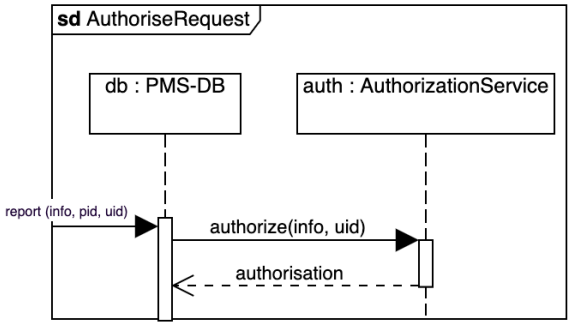
Also known as a Interaction Use.

- refers to another sequence diagram.
- Helps segment a diagram into manageable bits.



REF FRAGMENT

The fragment that is being referred to should be in a frame with an appropriate name.



HOW DOES THIS ALL FIT TOGETHER?

Use case diagram help us to work out what we are building, who is involved, what the requirements are and where to draw the line.

Class diagram allow us to model domain concepts and / or to create detailed plans.

Sequence diagram to specify or design interactions between things in details.

State machines help to understand the behavior of individual objects.

The same types of model are produced during the analysis and design phases but with differing levels of detail.

- Remember this is an iterative process.